

IHC – Practical Approach to Cyber Security (PACS)

MODULAR ASSIGNMENT 1

Threat Modeling and Security Architecture

Secure Cloud-Native Healthcare Appointment Booking System

Submitted by: Nitesh Vishwakarma

1. Assignment Objective

The objective is to design a secure, scalable cloud-native architecture for a healthcare appointment booking system and perform systematic threat modeling. The design covers functional and non-functional requirements, secure cloud architecture, STRIDE-based threat identification, risk prioritization, and practical mitigation controls.

2. System Overview and Assumptions

The proposed system is a cloud-native healthcare appointment booking platform. Patients register and book appointments; doctors manage availability and appointments; administrators manage users and configuration. The reference implementation uses a public cloud such as AWS, RESTful APIs, sensitive PII/health data, and HIPAA-related security considerations.

Area	Requirement / Assumption
Users	Patients, doctors and administrators.
Core functions	Registration, login, doctor availability, appointment booking, appointment management and administration.
Architecture	Public cloud deployment using AWS as the reference implementation.
Communication	RESTful APIs with HTTPS/TLS.
Data	PII and health-related information are treated as sensitive.
Compliance	HIPAA-related controls are considered; actual compliance requires organizational/legal validation.
Security	Least privilege, RBAC, encryption, logging, monitoring and secure API design.

3. Secure Cloud Architecture

Secure Cloud-Native Healthcare Appointment Platform

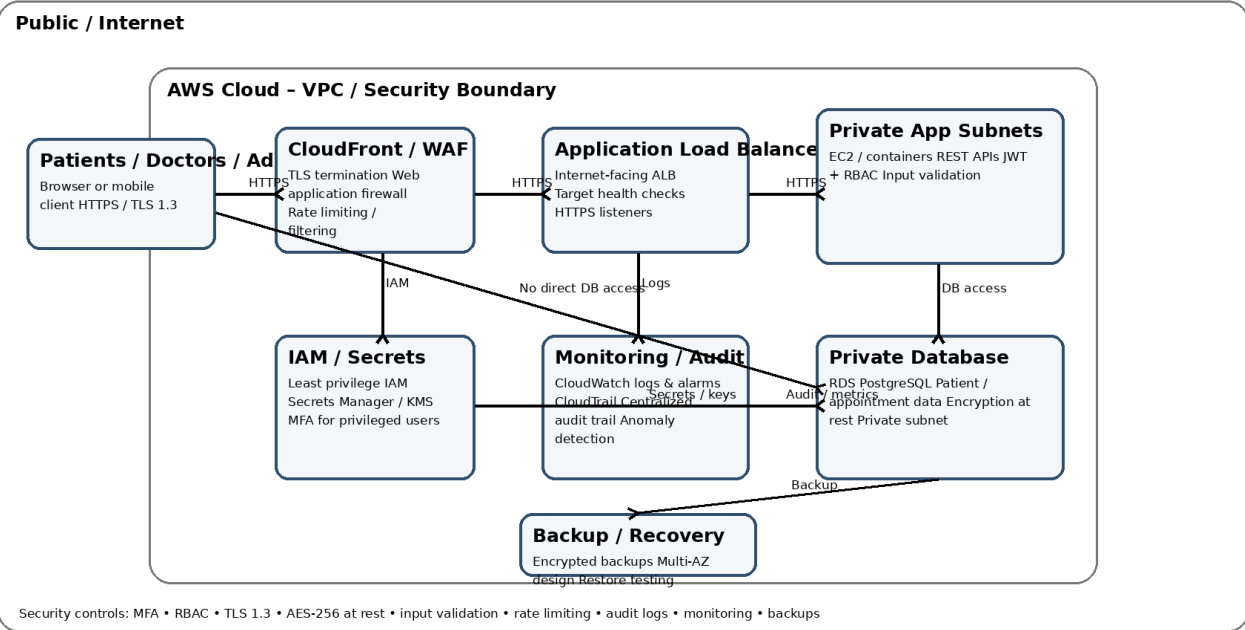


Figure 1. Proposed secure cloud-native architecture.

4. STRIDE Threat Model

STRIDE	Threat	Affected Component	Mitigation
Spoofing	Attacker impersonates a patient, doctor or administrator.	Authentication / identity layer	MFA for privileged users; strong authentication; secure token validation; account protection.
Tampering	Unauthorized modification of appointment or user data.	API Gateway / application / database	Input validation; authorization; integrity controls; controlled database permissions; audit logs.
Repudiation	A user denies having performed a security-sensitive action.	Backend / audit layer	Immutable or protected audit logs; timestamps; actor identity; centralized monitoring.
Information Disclosure	PII or health-related data is exposed.	Database / APIs / logs	TLS; encryption at rest; least privilege; private subnets; data minimization; careful log redaction.
Denial of Service	Attacker overwhelms APIs or infrastructure.	Edge / API / application	Rate limiting; WAF controls; autoscaling; health checks; monitoring and alerting.
Elevation of Privilege	Patient or low-privilege user accesses administrator functions.	Backend / authorization	RBAC; server-side authorization on every protected endpoint; least privilege; privileged MFA.

5. Proposed Risk Assessment

Risk	Likelihood	Impact	Priority	Primary Controls
Information disclosure of health/PII data	High	High	Critical	Encryption, least privilege, private DB, TLS, logging, data minimization.
Privilege escalation to administrator	Medium	High	High	RBAC, server-side authorization, MFA, audit logs.

Risk	Likelihood	Impact	Priority	Primary Controls
Account spoofing / credential compromise	Medium	High	High	Strong authentication, MFA for privileged roles, token controls.
Appointment data tampering	Medium	High	High	Input validation, authorization, integrity controls, audit trail.
API / service DoS	Medium	High	High	Rate limiting, WAF, autoscaling, monitoring.
Repudiation of sensitive actions	Medium	Medium	Medium	Centralized audit logging and protected timestamps.

6. Mitigation Strategies

Identity and Access Management: Use separate roles for patients, doctors and administrators. Apply least privilege to application and cloud identities. Require MFA for privileged accounts.

Encryption: Use TLS for data in transit and strong encryption such as AES-256-class controls for data at rest. Protect encryption keys separately and restrict key access.

API Security: Validate all inputs, authenticate protected requests, enforce authorization on the server, use secure token validation, rate-limit sensitive endpoints and avoid exposing internal errors.

Network Security: Place application servers and databases in private subnets. Use security groups/firewall rules to allow only required flows.

Logging and Monitoring: Centralize application and infrastructure logs, monitor authentication and authorization failures, create alerts for suspicious behavior and maintain audit records.

Compliance and Privacy: Minimize collected data, restrict access to sensitive records, maintain audit trails, define retention rules and document organizational HIPAA responsibilities.

Availability: Use load balancing, health checks, autoscaling, controlled rate limiting, backups and recovery procedures.

7. Authentication and Authorization

Authentication establishes identity, while authorization determines whether that identity may perform the requested operation. Protected APIs should validate tokens and apply server-side RBAC on every protected operation. Secrets should be stored in managed secret/key services rather than hard-coded.

8. Threat-to-Control Traceability

Threat	Prevent / Reduce	Detect / Evidence
Spoofing	Authentication, MFA, secure token validation	Identity logs; failed-login alerts
Tampering	Input validation, authorization, integrity controls	Application audit logs; change records
Repudiation	Protected audit logging	Timestamped actor/action records
Information Disclosure	TLS, encryption at rest, private DB, RBAC	Access logs; data-access monitoring
DoS	Rate limiting, WAF, autoscaling	Traffic metrics; anomaly alerts
Elevation of Privilege	RBAC, least privilege, privileged MFA	Authorization failures; IAM audit logs

9. Testing and Validation Plan

Test	Expected Result
Unauthenticated API request	Protected endpoint denies access and exposes no sensitive data.
Patient calls admin endpoint	Server-side authorization denies the request.
Invalid input	Request is rejected safely.
Excessive API requests	Rate limiting/edge controls reduce abusive traffic.
Internet-to-database connection	Direct access is prevented by private network placement.

Test	Expected Result
Sensitive data inspection	Stored sensitive data is encrypted and logs avoid unnecessary PII.
Audit event	Security-sensitive action generates traceable audit evidence.
Failure test	Unhealthy application targets are removed and traffic uses healthy targets.

10. Design Rationale

The architecture uses layered security: controlled public entry points, protected application and database tiers, RBAC, encryption, monitoring, auditability, rate limiting and resilience. Each control maps to one or more STRIDE threats and to the assignment's cloud-security principles.

11. Deliverables Checklist

- Architecture diagram
- Threat model document
- System overview and assumptions
- STRIDE table
- Risk assessment and prioritization
- Mitigation strategies
- Design rationale

12. Conclusion

The proposed design addresses the assignment's four rubric areas: system understanding, secure architecture and controls, threat identification/modeling, and mitigation/documentation. It provides a practical defense-in-depth approach for a cloud-native healthcare appointment platform.

References

OWASP Threat Modeling Project – <https://owasp.org/www-project-threat-modeling/>

Microsoft Threat Modeling Tool – <https://www.microsoft.com/en-us/securityengineering/sdl/threatmodeling>

Sample Healthcare Dataset – <https://www.kaggle.com/datasets/rohitro/healthcare-dataset-stroke-data>

IHC – Practical Approach to Cyber Security (PACS), Modular Assignment 1.